개인 회고

진행하면서 막혔던 부분과 해결

막힌 부분	원인	해결
내 노트북에서 할지 코랩에서 할지 고민	데이터셋만 약 1.9GB(압축을 풀면 더 늘어남)이고, PaDiM은 학습 특징 `[630336, 1536]`을 GPU에 모으는 것만 약 3.9GB, 합칠 때는 두 배 가까이 필요하다. 내 노트북(GPU 8GB)으로 했으면 용량도 많이 차지하고 시간도 한참 걸렸을 것 같았다	코랩 Pro(T4 GPU)에서 했다. 원본이 리눅스 기준 코드라 코랩에서는 거의 그대로 돌아갔다
첫 셀부터 `import wget`에서 막힐 뻔함	원본은 `# !pip install wget`까지 주석 처리되어 있는데, 코랩에는 `wget` 패키지가 없다(실행해 보니 새로 설치됐다)	이 줄만 주석을 풀고, 바로 위에 원본 줄과 바꾼 이유를 주석으로 남겼다. 원본 코드 중 바꾼 곳은 이 한 줄뿐이다

실습을 통해 알게 된 것

이상 탐지 이론을 배우는 데서 멈추지 않고 직접 구현해 보면서, 학습에는 정상 데이터만 쓰는데도 불량 데이터가 왜 필요한지 알게 됐다. 불량 데이터는 학습이 아니라, 다 만든 모델이 불량을 잘 가려내는지 시험하는 테스트 단계에서 필요하다.

또 이상 탐지는 목적에 따라 재현율(불량을 놓치지 않는 것)과 정밀도(불량이라고 한 것이 진짜 불량인 것) 중 무엇을 높일지 골라야 한다는 것을 알았다. 실습 코드의 평가 도구는 F1이 가장 높은 임계값(PaDiM은 22.26)을 찾아 둘의 균형을 맞췄다. 반면 개인 실험에서 웹 판정 기준인 임계값 15로 테스트셋을 판정해 보니, 실제 불량 100장은 모두 잡아 재현율은 100%였지만 실제 정상 201장 중 135장도 불량으로 판정해서 정밀도가 크게 떨어졌다(불량 판정 235장 중 진짜 불량은 100장).

세 모델은 정상만 보고 "평소와 다른 정도"를 점수로 매긴다는 점은 같고, 점수를 매기는 방식이 다르다. PaDiM은 사진을 바둑판처럼 칸으로 나눠서 칸마다 "여기는 보통 이렇게 생겼다"를 외워 두고, 가장 이상한 칸의 점수를 사진의 점수로 쓴다. One-Class SVM은 사진 전체를 숫자 묶음 하나로 요약한 뒤 정상들을 감싸는 울타리를 치고, 울타리 밖이면 이상으로 본다. AutoEncoder는 사진을 작게 접었다가 다시 펴는 연습을 정상 사진으로만 시켜서, 잘 못 펴지는 사진을 이상으로 본다. 이번 실습에서는 위치까지 보는 PaDiM이 가장 잘했다(AUROC 0.962).

어려웠던 것

AutoEncoder의 AUROC가 0.26이 나와서 처음에는 그냥 성능이 나쁜 줄 알았다. 그런데 공부해 보니 AUROC가 0.5보다 낮다는 것은 아무것도 모르는 게 아니라 정상과 불량을 거꾸로 맞혔다는 뜻이었다. 그래서 점수를 뒤집으면 1 - 0.264 = 약 0.74가 된다. 다만 뒤집어도 PaDiM(0.962)이나 One-Class SVM(0.824)보다 낮고, 원래 원인은 출력 범위(0~1)가 입력 범위(약 -2.1~2.6)와 달라 제대로 복원할 수 없는 구조라서, 뒤집는다고 좋은 모델이 되는 것은 아니다.

좋았던 점과 아쉬웠던 점

코랩으로 진행해서 학습이 빨리 돼 좋았다.

(아쉬웠던 점: 답변 대기)

처음 기대와 실제

(Q6 답변 대기)

현장에 쓴다면

실제 공장에서 쓴다면 여러 임계값으로 테스트해 보고, 원하는 정밀도와 재현율을 확보하는 값을 고를 것 같다. 임계값 15처럼 정해 두면 실제 정상의 67%를 불량으로 판정하게 된다. 또 정상 제품이 불량으로 잘못 판정된 사진을 살펴보니 기판이 기울어져 촬영된 사진이 있었고, 위치마다 정상을 외우는 PaDiM에는 그것만으로도 이상해 보였을 가능성이 크다. 그래서 데이터를 모을 때부터 제품을 같은 위치·각도로 놓고 일관된 형태로 촬영할 것 같다.

앞으로 해 볼 것

PaDiM은 차원 축소 행렬, 좌우 뒤집기, 순서 섞기가 무작위라 실제로 두 번 돌렸을 때 AUROC가 0.961, 0.962로 달랐다. 랜덤 시드를 5~10개 정도 바꿔 돌려 보고 평균을 내서, 모델의 무작위성에서 오는 오차를 통제해 보고 싶다.

동료 피드백

(Q9 답변 대기)